

Indian Monument Scavenger Hunt using CLIP*

Team - Stress, Math and AI

Amisha Jindal
IIIT Hyderabad
Hyderabad, Telangana, India
amisha.amisha@research.iiit.ac.in

Ananya Kasavajhala
IIIT Hyderabad
Hyderabad, Telangana, India
ananya.kasavajhala@research.iiit.ac.in

Harsh Kapoor
IIIT Hyderabad
Hyderabad, Telangana, India
harsh.kapoor@research.iiit.ac.in

Udith Krishna
IIIT Hyderabad
Hyderabad, Telangana, India
udithkrishna.nair@research.iiit.ac.in

Vedant Tejas
IIIT Hyderabad
Hyderabad, Telangana, India
vedant.tejas@research.iiit.ac.in

ABSTRACT

This project presents the design and implementation of a Monument Scavenger Hunt application, a gamified heritage identifier designed to enhance the tourist experience through computer vision. The application is built upon a Streamlit framework, allowing users to identify monuments via photo uploads and receive immediate historical context, visiting hours, and ticketing information. The system leverages OpenAI's CLIP (Contrastive Language-Image Pre-training) architecture, transitioning from a zero-shot baseline to a fine-tuned model to improve classification accuracy across 30+ Indian monument classes and various regional sub-categories. A primary focus of this work is the analysis of model confusion, particularly addressing the visual similarities inherent in Indian architectural styles, such as Dravidian temples and Maratha forts. The app further incorporates a scavenger hunt mode featuring LLM-generated clues, creating an immersive loop of discovery and vision-based verification.

CCS CONCEPTS

• **Computing methodologies** → **Computer vision**; *Transfer learning*; • **Applied computing** → *Arts and humanities*; Travel and tourism.

KEYWORDS

Computer Vision, CLIP, Fine-tuning, Indian Heritage, Scavenger Hunt, Model Confusion Analysis, Transfer Learning

1 INTRODUCTION

The preservation and exploration of cultural heritage have traditionally relied on passive forms of engagement. Tourists visiting historical landmarks are often limited to reading static physical placards, listening to rigid audio guides, or relying on manual internet searches, a process that inherently assumes the user already knows the name or specific context of the monument they are viewing. This creates a friction point in the tourist experience, separating the physical act of exploration from the acquisition of historical knowledge. This project, the Monument Scavenger Hunt, proposes

a paradigm shift: moving from passive observation to active, **gamified engagement through the application of state-of-the-art computer vision and multi-modal artificial intelligence**.

From a machine learning perspective, building a highly accurate, scalable monument identifier presents significant challenges. Historically, this task has been approached using standard Convolutional Neural Networks (CNNs) like ResNet or VGG, trained on closed-set categorical labels. However, closed-set classifiers are extremely rigid. Furthermore, Indian architecture presents a unique challenge known as high intra-class visual similarity. For instance, numerous Maratha forts in Maharashtra share identical high-altitude topographies and black basalt stone construction, while various Dravidian temples exhibit nearly identical *gopuram* structures. Traditional models frequently fail to capture the fine-grained discriminative features required to separate these classes.

To overcome these scalability and accuracy limitations, our system is built upon OpenAI's Contrastive Language-Image Pre-training (CLIP) architecture. By mapping both images and natural language text into a shared, high-dimensional latent space, CLIP can *perform zero-shot classification*. It does not predict an integer class label; rather, it identifies which text prompt exhibits the highest cosine similarity to the uploaded image's visual embedding. This allows our application to dynamically expand its recognizable vocabulary simply by updating a text list, without requiring structural changes to the model.

While CLIP provides a robust zero-shot baseline, this report details our methodology for pushing the architecture further via light fine-tuning. We demonstrate how targeted contrastive training on a curated dataset of Indian monuments allows the Vision Transformer (ViT) to adapt its attention mechanisms specifically to the textures and structural nuances of Indian heritage.

The primary contributions of this project are as follows:

- (1) **Gamified Application Development**
- (2) **Multi-Modal Fine-Tuning**
- (3) **Comprehensive Confusion Analysis**

The remainder of this report is organized as follows: Section 2 details the data processing pipeline. Section 3 outlines CLIP model architecture. Section 4 describes our methodology. Section 5 presents the app interface and system flow. Section 6 provides our results and analysis, followed by ablation studies in Section 7. Finally, Section 8 discusses current limitations and avenues for future work.

*This paper is submitted in partial fulfillment of the project requirements for the Statistical Methods in AI (SMAI) course, Spring 2026, at the International Institute of Information Technology (IIIT), Hyderabad.

2 DATA ACQUISITION & PREPARATION

The efficacy of a multi-modal foundational model like CLIP depends heavily on the quality, diversity, and contextual accuracy of both the visual and textual data. To train our Low-Rank Adaptation (LoRA) modules effectively across 50 distinct Indian monuments, we structured our data preparation into parallel pipelines: visual curation, text prototype construction, and metadata aggregation.

2.1 Data Sources & Collection Methodology

Rather than relying on a single monolithic database, we aggregated data from four distinct primary sources to ensure comprehensive coverage of the 50 selected monument classes:

- **Kaggle Repository (Visual Baseline):** The foundational visual data was sourced from the publicly available "Indian Monuments Image Dataset" [6]. This provided an initial bulk download of approximately 3,500 pre-categorized images covering globally recognized core classes (e.g., Taj Mahal, Charminar).
- **Wikimedia Commons Scraping (Visual Augmentation):** To expand our dataset to 50 distinct classes and introduce challenging regional heritage sites (such as specific Maharashtra forts and Indian stepwells), we engineered a custom web-scraping pipeline targeting Wikimedia Commons [7]. By querying specific architectural category URLs, we programmatically downloaded an additional 884 high-resolution images.
- **Wikipedia & Open-Source APIs (Logistical Metadata):** For the structured data required by the Streamlit application's user interface, we scraped the official Wikipedia pages corresponding to each of the 50 monuments. We systematically extracted precise geospatial coordinates (latitude and longitude for Google Maps routing), historical summaries, and current logistical data (visiting hours and ticketing tiers).
- **LLM Generation (Gamification Clues):** To fuel the "Scavenger Hunt" verification mode, we required obscure, engaging trivia that could not be easily reverse-engineered by a simple Google search. We utilized Large Language Models (specifically Google Gemini) via prompt engineering to generate 3 to 5 unique fun_facts for each monument.

All collected data was manually verified for historical accuracy and compiled into a centralized, offline knowledge graph (`monuments_metadata.json`). With the raw multimodal data successfully aggregated, the next critical step was standardizing the visual inputs for the neural network.

2.2 Visual Dataset Curation & Preprocessing

To ensure the vision encoder generalizes well to the noisy conditions of real-world tourist photography (e.g., varying lighting, smartphone camera quality, and unpredictable angles), the input RGB images underwent a strict preprocessing and augmentation pipeline. All images were resized and center-cropped to fit the 224×224 input dimensionality expected by the ViT-B/32 architecture. During the fine-tuning phase, we applied stochastic augmentations including random cropping and color jittering (variations in brightness, contrast, and saturation).

2.3 Textual Prototypes & Prompt Engineering

For each of the 50 monument classes, we precomputed text features using the frozen text encoder, averaged them, and normalized them to create fixed **Text Prototypes**.

We experimented with three distinct prompt engineering strategies to construct these prototypes:

- (1) **Baseline Prompts:** The simplest approach, utilizing a single standard template (e.g., "A photo of the Taj Mahal.").
- (2) **Ensemble Prompts:** A collection of 25 generic architectural and photographic templates (e.g., "The iconic {} ", "A nighttime photo of the {} illuminated by lights."). The text features extracted from all 25 prompts are mathematically averaged and normalized into a single, robust vector per monument.
- (3) **Custom Ensemble Prompts:** Highly detailed, visually descriptive sentences crafted specifically for each of the 50 monuments (stored centrally in `custom_prompts.py`). These descriptions capture unique architectural elements, materials, and historical context (e.g., "An architectural detail shot of the intricate pietra dura inlay and calligraphy on the Taj Mahal.").

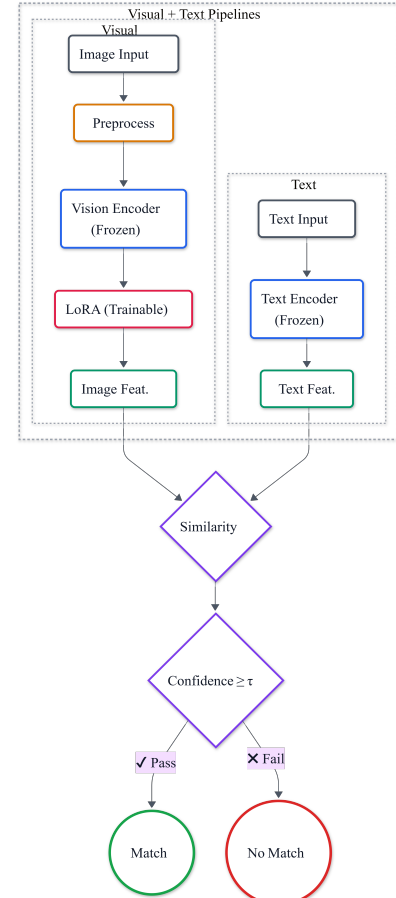


Figure 1: DataSet Creation and Processing

3 MODEL ARCHITECTURE: CLIP & LORA

To process the parallel streams of visual and textual data curated in the previous stage, our system leverages the Contrastive Language-Image Pre-training (CLIP) architecture [1]. Specifically, we utilize the openai/clip-vit-base-patch32 variant. CLIP represents a fundamental departure from traditional closed-set categorical classifiers; by learning a shared, multi-modal embedding space, it enables open-vocabulary verification, which is structurally ideal for a dynamic scavenger hunt application.

To bypass the computationally prohibitive cost and catastrophic forgetting associated with full fine-tuning, we employed Parameter-Efficient Fine-Tuning (PEFT) [4] via Low-Rank Adaptation (LoRA) [3].

3.1 Vision and Text Encoders

The core architecture consists of two distinct neural network branches:

- **Vision Encoder (ViT-B/32):** The visual branch processes augmented photographs using a Vision Transformer (ViT) [2]. An input image $x_v \in \mathbb{R}^{H \times W \times C}$ (where $H = 224$, $W = 224$) is divided into non-overlapping patches of size 32×32 . A learnable [CLS] (classification) token is prepended to the sequence, which is then passed through 12 Transformer encoder blocks. The state of the [CLS] token at the output of the final layer serves as the global image representation, denoted as $f_I \in \mathbb{R}^d$ (where $d = 512$).
- **Text Encoder:** The text branch relies on a standard decoder-only Transformer. The prompt string is tokenized using byte-pair encoding (BPE). The sequence is passed through 12 Transformer layers, and the activation of the highest layer at the [EOS] (End of Sequence) token is extracted and linearly projected into the shared d -dimensional space, yielding the text representation $f_T \in \mathbb{R}^d$.

In our architectural design, the base weights of both the ViT and the Text Encoder are kept **strictly frozen**. Feature adaptation is handled exclusively by the downstream LoRA modules.

3.2 Low-Rank Adaptation (LoRA) Injection

To adapt the Vision Encoder to the fine-grained nuances of Indian architecture without overfitting, we inject trainable LoRA matrices specifically into the Query (W_q) and Value (W_v) projection layers of the ViT's self-attention blocks.

For a pre-trained, frozen weight matrix $W_0 \in \mathbb{R}^{d_{in} \times d_{out}}$, LoRA constrains its parameter update by representing it with a low-rank decomposition:

$$W = W_0 + \Delta W = W_0 + BA \quad (1)$$

where $B \in \mathbb{R}^{d_{in} \times r}$ and $A \in \mathbb{R}^{r \times d_{out}}$, and the rank $r \ll \min(d_{in}, d_{out})$. We selected a rank of $r = 32$ and a scaling factor of $\alpha = 32$. During the forward pass, the modified activation h for an input x is computed as:

$$h = W_0 x + \frac{\alpha}{r} B A x \quad (2)$$

This allows the model to adapt highly specific architectural attention maps while updating fewer than 1% of the total parameters, ensuring the foundational generalizability of CLIP remains intact.

3.3 Contrastive Verification Engine

Traditional monument identification treats the problem as a 1:N Classification Task. However, our Scavenger Hunt relies on a **1:1 Verification Task**: the application knows the target monument it challenged the user to find, and it must verify if the user's uploaded photo matches that specific target prototype.

To achieve this authentication, the engine calculates the cosine similarity between the LoRA-adapted image embedding $f_I^{LoRA}(x)$ and the precomputed target text prototype t_{target} :

$$\text{sim}(f_I^{LoRA}(x), t_{target}) = \frac{f_I^{LoRA}(x) \cdot t_{target}}{\|f_I^{LoRA}(x)\| \|t_{target}\|} \quad (3)$$

This raw similarity is scaled by a learned temperature parameter (τ) to produce a final confidence score S :

$$S = \exp\left(\frac{\text{sim}(f_I^{LoRA}(x), t_{target})}{\tau}\right) \quad (4)$$

The verification engine functions as a strict gateway governed by an empirical confidence threshold, τ_{min} . If $S \geq \tau_{min}$, the application authenticates the match. If $S < \tau_{min}$, the photo is securely rejected as a mismatch or spoofing attempt.

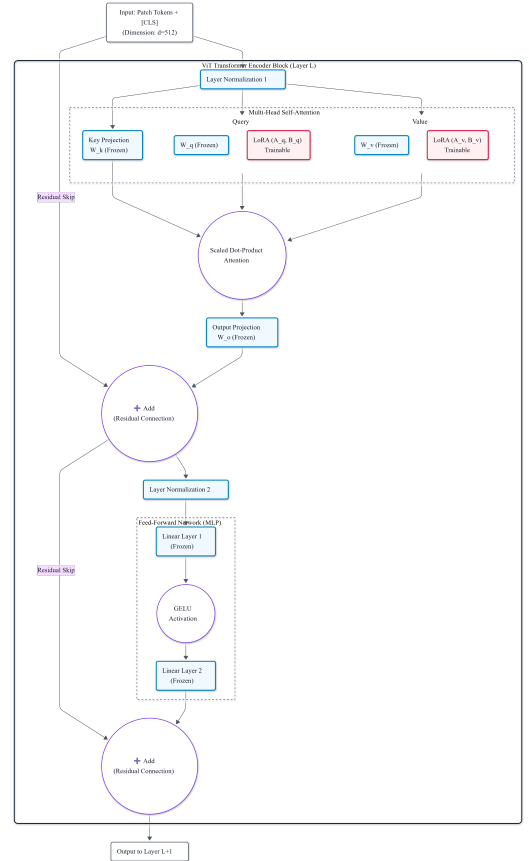


Figure 2: ViT Transformer Encoder Block

4 METHODOLOGY & EVALUATION FRAMEWORK

To rigorously benchmark the efficacy of the LoRA-adapted Vision Encoder, we designed a comprehensive evaluation framework. We recognize that standard Top-1 accuracy is insufficient to measure the robustness of a deployed mobile application. Therefore, our methodology forces the model to be analyzed through two completely distinct mathematical lenses: Classification (measuring foundational intelligence) and Verification (measuring production readiness).

4.1 Task 1: Classification

The classification task evaluates the model’s fundamental capacity to distinguish between highly similar architectural classes. It addresses the core question: *"Out of all 50 Indian monuments, which one does this photo resemble the most?"*

Mathematically, this is framed as a $1 : N$ retrieval task. Given a LoRA-adapted image embedding $f_I(x)$, we compute the cosine similarity against the entire set of 50 frozen text prototypes $\mathcal{T} = \{t_1, t_2, \dots, t_{50}\}$.

The probability distribution over the classes is defined by the softmax function applied to the scaled similarities:

$$P(y = c|x) = \frac{\exp(\text{sim}(f_I(x), t_c)/\tau)}{\sum_{j=1}^{50} \exp(\text{sim}(f_I(x), t_j)/\tau)} \quad (5)$$

where τ is a learned temperature parameter that controls the sharpness of the distribution.

During the fine-tuning phase, the LoRA adapters are trained using a symmetric Cross-Entropy Loss (often referred to as InfoNCE loss in contrastive learning [5]) applied to these logits, explicitly maximizing $P(y = c_{\text{correct}}|x)$.

For evaluation, we rely on **Top-1 Accuracy** and detailed **Confusion Matrices** to pinpoint exact class overlaps (e.g., differentiating between structurally similar Maharashtra forts). While crucial for proving the model learned the underlying architecture, classification logic is fundamentally flawed for the gamified scavenger hunt. Because probabilities must sum to 1.0, the softmax function will erroneously force the model to identify a random, out-of-distribution photo (e.g., a street dog) as the "most likely" monument, breaking the game’s security.

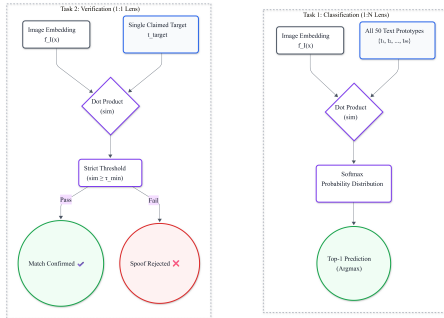


Figure 3: Comparison of the evaluation tasks

4.2 Task 2: Verification

To mirror the strict logic of the Streamlit Scavenger Hunt application, we transitioned our primary evaluation to Verification. This task addresses the question: *"The user claims this is a photo of Target X. Is this claim mathematically valid?"*

Verification reformulates the complex 50-class problem into an isolated, binary "Yes/No" gateway. We discard the softmax function entirely. Instead, we compute the isolated cosine similarity score S strictly between the uploaded image $f_I(x)$ and the known target prototype t_{target} . The decision function $\hat{y}$ is governed by an empirical confidence threshold τ_{min} :

$$\hat{y} = \begin{cases} 1 & \text{if } S \geq \tau_{\text{min}} \\ 0 & \text{if } S < \tau_{\text{min}} \end{cases} \quad (6)$$

This methodology is evaluated using biometric authentication metrics:

- **ROC AUC Score:** The Area Under the Receiver Operating Characteristic curve, measuring how perfectly the model separates genuine photos (True Positives) from impostor/incorrect photos (False Positives).
- **Equal Error Rate (EER):** The exact operating threshold where the False Accept Rate (FAR) equals the False Reject Rate (FRR).
- **TAR @ FAR:** The True Accept Rate at a strictly bounded False Accept Rate. By evaluating TAR @ 1.0% FAR, we simulate a "Hard Mode" for the game (highly secure against cheating, but requiring pristine photos). Evaluating TAR @ 5.0% FAR simulates "Easy Mode" (forgiving of poor lighting or smartphone blur, at the cost of slight spoofing vulnerability).

4.3 The 4x3 Evaluation Matrix

To systematically quantify the impact of our prompt engineering and PEFT strategies, we executed a rigorous 4×3 evaluation matrix across both the Classification and Verification tasks.

We tested four distinct model states:

- (1) Base CLIP (Zero-shot, frozen weights, no LoRA adapters)
- (2) Baseline LoRA ($r = 32$, trained on standard templates)
- (3) Ensemble LoRA ($r = 32$, trained on averaged generic templates)
- (4) Custom Ensemble LoRA ($r = 32$, trained on highly descriptive, monument-specific vectors)

Each of these four models was cross-evaluated against three distinct test prompt types (Baseline, Ensemble, and Custom). This matrix yielded 12 distinct evaluation reports for classification and 12 for verification, allowing us to definitively isolate the performance gains attributed directly to the LoRA fine-tuning versus the gains attributed to superior prompt engineering.

5 DEPLOYMENT: STREAMLIT UI

To transition our machine learning models from backend tensors to a production-ready user experience, we developed a frontend application titled *Monument Explorer*. Built using the Streamlit framework, the application utilizes a sidebar routing architecture to offer

users two distinct operational modes: a traditional identification tool and a gamified exploration mechanic.

5.1 Mode 1: Direct Classification

The "Direct Classification" mode serves as a straightforward 1:N diagnostic tool (as evaluated in Section 4.1). Users are presented with a file uploader to submit an image of any Indian heritage site.

Upon upload, the application pushes the image through the frozen ViT-B/32 encoder and the custom LoRA adapters. It calculates the softmax probability distribution across all 50 pre-computed text prototypes. The UI then dynamically renders the Top-1 Prediction alongside its absolute confidence score (e.g., "Very High confidence - 94.0%"). To provide interpretability, the application utilizes `st.progress` bars to display a horizontal chart of the top 5 nearest-neighbor predictions, allowing researchers to visually inspect the model's confidence distribution and potential class confusion.

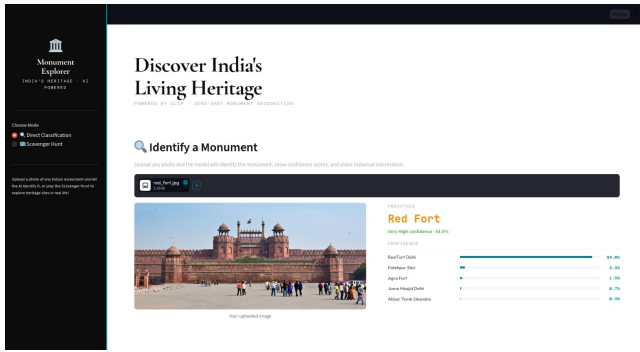


Figure 4: Direct Classification Mode. The UI displays the Top-1 prediction (Red Fort) alongside a distribution chart of competing confidence scores (e.g., Fatehpur Sikri, Agra Fort).

5.2 Mode 2: Scavenger Hunt (Finite State Machine)

The "Scavenger Hunt" mode represents the core objective of this project: a 1:1 vision-verified game. Because Streamlit inherently re-runs the entire script upon any user interaction, we architected this mode as a Finite State Machine (FSM) utilizing `st.session_state` to preserve game progression.

5.2.1 State 1: Challenge Initialization. The user begins by selecting a geographic region (e.g., "Madurai, Tamil Nadu") via a dropdown interface. Rather than immediately revealing the target, the application queries the localized JSON knowledge graph and initiates a progressive clue system. Using Streamlit's expander widgets, the UI reveals sequential, LLM-generated hints ranging from broad geographic indicators to specific architectural styles (e.g., "Hindu Temple with Dravidian Architecture").

5.2.2 State 2: Capture and Inference. Once the user believes they have physically located the monument based on the clues, they upload a photograph. To guarantee zero-latency UI responsiveness, the 50 Text Prototypes are cached offline as a static PyTorch tensor.

The system executes the 1:1 Verification logic, computing the cosine similarity exclusively against the hidden target's text prototype.

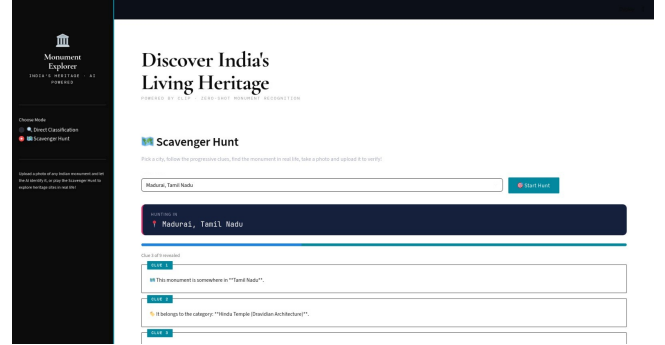


Figure 5: Scavenger Hunt Initialization. The user selects a geographic region and is presented with progressive, gamified textual clues extracted from the JSON metadata graph.

5.2.3 State 3: Resolution and Reward. The Streamlit router evaluates the resulting similarity score against the strict confidence threshold τ_{min} :

- **Mismatch:** If the similarity falls below τ_{min} , the app triggers an `st.error` warning, notifying the user that the image is incorrect and loops them back to the upload state without revealing the answer.
- **Match Verified:** If the similarity surpasses τ_{min} , the FSM transitions to the resolution state. The system displays a success banner ("Congratulations! You found Meenakshi Amman Temple!") along with the verification confidence score.

Upon a successful verification, the state machine unlocks the hidden metadata dashboard. The application dynamically renders the "About this Monument" section, pulling detailed historical summaries, exact geographical locations, and logistical data directly from the JSON graph, thereby completing the educational loop of the gamified experience.

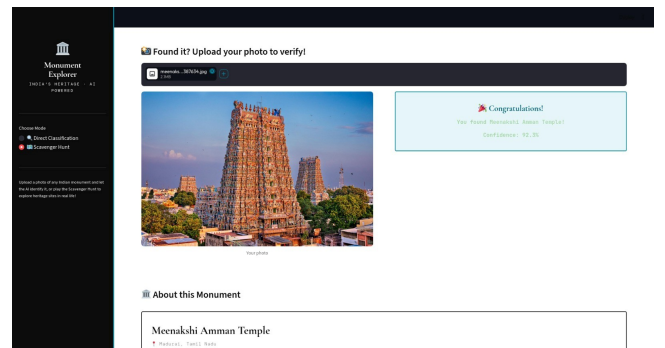


Figure 6: Scavenger Hunt Resolution. Upon uploading a verified image, the system surpasses the τ_{min} threshold, unlocking the success banner and the detailed historical dashboard.

6 RESULTS AND ANALYSIS

6.1 Classification Performance

The classification task evaluates the model’s fundamental ability to correctly identify the monument in a given image out of the 50 possible classes. Table 1 details the overall Top-1 accuracy metrics across all fine-tuning configurations and prompting strategies.

Table 1: Classification Accuracy Matrix (Top-1 %)

Model Architecture	Baseline Prompts	Ensemble Prompts	Custom Ensemble
Base CLIP (Frozen)	65.15%	65.36%	66.80%
Baseline LoRA	90.27%	90.34%	88.06%
Ensemble LoRA	88.89%	89.92%	88.13%
Custom Ensemble LoRA	83.23%	86.68%	92.34%

As observed, fine-tuning via LoRA yields a massive performance increase across all configurations. The *Custom Ensemble LoRA*, when utilizing its aligned *Custom Ensemble* prompt, achieved the peak aggregate accuracy of 92.34%.

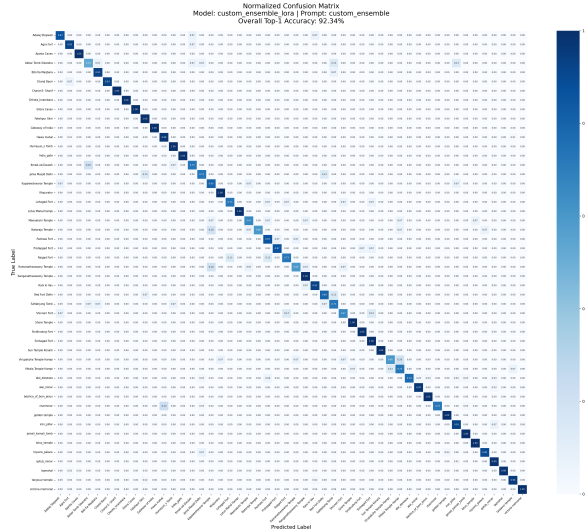


Figure 7: Confusion Matrix for the peak-performing model (Custom Ensemble LoRA with Custom Ensemble Prompts), illustrating the distribution of true vs. predicted classes and highlighting areas of architectural overlap.

6.2 Verification Performance

While classification identifies the most likely class, verification acts as the strict 1:1 authentication gateway for our application. It tests the model’s ability to discern whether an uploaded image legitimately matches the claimed target monument. This is evaluated via Equal Error Rate (EER), ROC Area Under the Curve (AUC), and True Accept Rate (TAR) at a strict 1.0% False Accept Rate (FAR).

6.3 Inferences and Discussion

The empirical evaluation yields several critical insights regarding architectural choices and prompting dynamics within the multimodal embedding space:

Table 2: Verification Metrics: Equal Error Rate (EER) [Lower is Better]

Model Architecture	Baseline Prompts	Ensemble Prompts	Custom Ensemble
Base CLIP (Frozen)	7.88%	7.50%	6.37%
Baseline LoRA	1.74%	1.88%	2.11%
Ensemble LoRA	1.67%	1.68%	2.03%
Custom Ensemble LoRA	2.76%	2.46%	1.79%

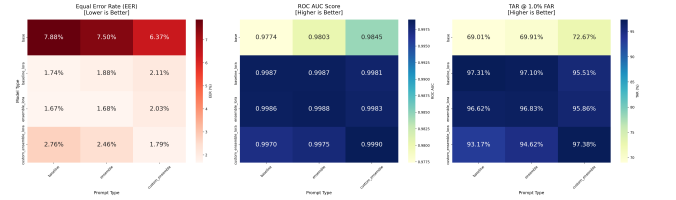


Figure 8: Receiver Operating Characteristic (ROC) curves comparing the baseline and fine-tuned models. The Custom Ensemble LoRA exhibits the highest Area Under the Curve, indicating superior class separability.

Table 3: Verification Metrics: ROC AUC Score [Higher is Better]

Model Architecture	Baseline Prompts	Ensemble Prompts	Custom Ensemble
Base CLIP (Frozen)	0.9774	0.9803	0.9845
Baseline LoRA	0.9987	0.9987	0.9981
Ensemble LoRA	0.9986	0.9988	0.9983
Custom Ensemble LoRA	0.9970	0.9975	0.9990

Table 4: Verification Metrics: True Accept Rate (TAR) @ 1.0% FAR [Higher is Better]

Model Architecture	Baseline Prompts	Ensemble Prompts	Custom Ensemble
Base CLIP (Frozen)	69.01%	69.91%	72.67%
Baseline LoRA	97.31%	97.10%	95.51%
Ensemble LoRA	96.62%	96.83%	95.86%
Custom Ensemble LoRA	93.17%	94.62%	97.38%

6.3.1 The Necessity of LoRA Fine-Tuning. The most immediate inference is the stark performance gap before and after fine-tuning. Across all evaluations, the application of Low-Rank Adaptation (LoRA) is strictly necessary to achieve production-grade metrics. Zero-shot classification accuracy uniformly jumped from the ~ 65% baseline range to a highly robust 88% – 92% range upon LoRA injection.

6.3.2 Prompt-Model Alignment Dependency. The data strongly indicates that the fine-tuned adapters are highly sensitive to the prompting strategies they were exposed to during training. A configuration mismatch leads to severe degradation. For example, the *Custom Ensemble LoRA* model achieves its 92.34% peak with the Custom Ensemble prompt, but drops significantly to 83.23% when queried with standard Baseline prompts. Alignment between training and inference prompts is an absolute necessity for optimal performance.

6.3.3 The Top Performer: Custom Ensemble LoRA. When evaluating peak operational capability, the *Custom Ensemble LoRA* (paired with matching custom prompts) emerges as the definitive optimal architecture. Specifically:

- It achieved the absolute highest classification accuracy (**92.34%**).
- It mapped the highest ROC AUC score (**0.9990**), demonstrating extreme class separability.
- It maintained the highest True Accept Rate (**97.38%**) under the heavily constrained condition of a 1.0% False Accept Rate, making it exceptionally resilient to spoofing attempts.

6.3.4 Trade-offs in Verification: Crossover Error Rates. While the Custom Ensemble approach excels at strict thresholds (i.e., operating at 1.0% FAR), the standard *Ensemble LoRA* combined with baseline prompts provided slightly superior generalized crossover metrics, achieving the lowest Equal Error Rate (EER) of **1.67%**. If the final deployment environment requires an exact balance between false accepts and false rejects rather than prioritizing extreme strictness, the standard Ensemble LoRA represents a highly stable, generalized alternative.

7 CONCLUSION

The project successfully demonstrates the efficacy of Parameter-Efficient Fine-Tuning (PEFT) through Low-Rank Adaptation (LoRA) for specialized architectural recognition. By freezing the foundational CLIP model and injecting trainable LoRA adapters with a rank of $r = 32$, we achieved a state-of-the-art Top-1 classification accuracy of 92.34%. Furthermore, our 1:1 verification engine, designed for the "Monument Explorer" scavenger hunt, exhibited exceptional robustness with a True Accept Rate of 97.38% at a strict 1.0% False Accept Rate. The empirical results confirm that while baseline multi-modal models provide a strong zero-shot foundation, targeted fine-tuning and meticulous prompt engineering—specifically through Custom Ensembles—are critical for production-grade deployment. This architecture not only preserves the generalizable intelligence of CLIP but also adapts it to the unique visual nuances of Indian heritage sites. The resulting Streamlit application serves as a scalable blueprint for interactive, vision-verified educational platforms that bridge the gap between AI research and real-world cultural exploration. We urge you to check the deployed app on Hugging Face[10] for the app, the Github Repository[9] for all the Documentations and the Dataset on Hugging Face[11].

REFERENCES

- [1] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. (2021). *Learning transferable visual models from natural language supervision*. In International Conference on Machine Learning (pp. 8748-8763). PMLR.
- [2] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al. (2020). *An image is worth 16x16 words: Transformers for image recognition at scale*. arXiv preprint arXiv:2010.11929.
- [3] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-rank adaptation of large language models*. arXiv preprint arXiv:2106.09685.
- [4] Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., & Paul, S. (2022). *PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods*. Hugging Face. Retrieved from <https://github.com/huggingface/peft>
- [5] Oord, A. V. D., Li, Y., & Vinyals, O. (2018). *Representation learning with contrastive predictive coding*. arXiv preprint arXiv:1807.03748.
- [6] Danushkumar V. (2023). *Indian Monuments Image Dataset*. Kaggle. Retrieved from <https://www.kaggle.com/datasets/danushkumarv/indian-monuments-image-dataset>
- [7] Wikimedia Foundation. (2026). *Wikimedia Commons: Free Media Repository*. Retrieved from <https://commons.wikimedia.org/>
- [8] Streamlit Inc. (2024). *Streamlit: The fastest way to build data apps in Python*. Retrieved from <https://streamlit.io/>
- [9] Github. *Github repository of the assignment at Link*
- [10] HFD. *HuggingFace deployed version of the StreamLit App Link*
- [11] HFDat. *Dataset on HuggingFace Link*